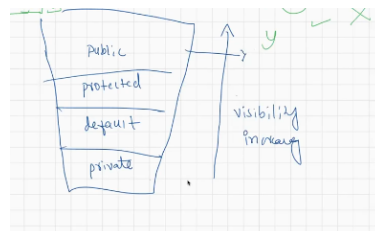


access specifiers

- public
- protected
- default
- private

can be used with class,method,variable

	within a class	outside class within package	→ package outside package (is-A relationship)	outside package no is-A relation
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default	✓	✓	✗	✗
private	✓	✗	✗	✗



the methods which are inherited from parents and used as it is are called inherited method

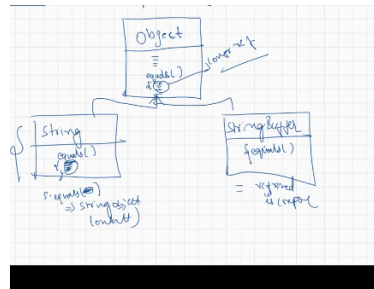
the method which are inherited from parents but modified as per the requirement of the child class are called overriding method

method which are in child class but not in parent class are called specified method

Symbolic representation of access modifiers

+	→ public
#	→ protected
~	→ default
-	→ private

example of method overridden in java core



Rules for overriding a method

- we cannot reduce visibility of overridden method but we can increase (example in parent class its public and u did private in child class this is not allowed)
- return type of overridden method must be same of the method which got override
- return type of the overridden method in child class can be different as that of parent if it is covariant return type (is a relationship)
- parameters of overridden method must be same as of parent else it will be considered as specialised method considering method overloading
- final class does not participate in inheritance
- but final method indeed do participate however we can't override the method
- Final variable act as constant we can't change them

private static method can't be overridden

super vs super()

super -> keyword

this() & this => (L)

super();

class Demo1

{
int age = 26;
}

}

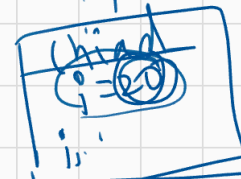
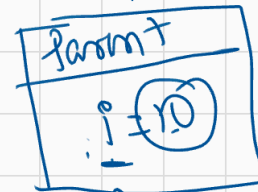
& class Demo2 extends Demo1

{
int age = 28;
}

void disp()

{
s.o.p(age);
// super.age;
}

28 =



i -> 20

super.i -> 10

super, super.i -> error



final keyword

class → To create class

|||||

new
↓
to
create
obj c

① final → class

final ⇒ class
⇒ method
⇒ variable

Abstract

```

Q>
class A {
    public String toString() {
        return null;
    }
}
public class Test {
    public static void main(String [] args) {
        String text = null;
        text = text + new A(); //Line n1    // JVM  text = null + "null"  text =
"nullnull"
        System.out.println(text.length()); //Line n2
    }
}
A. Line n1 causes compilation error
B. Line n1 causes Runtime error
C. Line n2 causes RunTime error
D. 0
E. 4
G. 8

```

Answer: G

Consider below code:

```

//Test.java
class SpecialString {
    String str;
    SpecialString(String str) {
        this.str = str;
    }
}
public class Test {
    public static void main(String[] args) {
        Object [] arr = new Object[4];
        for(int i = 1; i <=3; i++) {
            switch(i) {
                case 1:
                    arr[i] = new String("Java");
                    break;
                case 2:
                    arr[i] = new StringBuilder("Java");
                    break;
                case 3:
                    arr[i] = new SpecialString("Java");
                    break;
            }
        }
        for(Object obj : arr) {
            System.out.println(obj);
        }
    }
}

```

What will be the result of compiling and executing Test class?

- A. Java
Java
Java
- B. Java
Java

Some text with @symbol
 C. Java
 Some text with @symbol
 Some text with @symbol
 D. null
 Java
 Java
 Java
 Java
 E. null
 Java
 Java
 Some text with @symbol
 F. null
 Java
 Some text with @symbol
 Some text with @symbol
 G. Java
 Java
 Java
 null
 H. Java
 Java
 Some text with @symbol
 null
 I. Java
 Some text with @symbol
 Some text with @symbol
 null

Answer: E

```
Q> .
class MyStringClass extends String
{
    String name;
}
```

Output: CompileTime Error

```
Q>
String name = "sachinrameshtendulkar".substring(4);
System.out.println(name); //inrameshtendulkar
```

```
Q> .
String s = "1".repeat(5);
System.out.println(s); //11111
```

```
Q> .
System.out.println("1".concat("2").repeat(5).charAt(7));
1212121212.charAt(7) - > 2
```

Q>
 To which of the following classes, you can create objects without using new operator?

String
 StringBuffer
 StringBuilder
 Answer: String

Q> .
String string = "string".replace('i', '0');
System.out.println(string.substring(2, 5));

string = "str0ng";
output: r0n

Q> .
In my application, I want mutable and thread safe string objects. Which class do you refer me to use? String or StringBuffer or StringBuilder?
Answer StringBuffer(synchronized)

Q> .
System.out.println("Java" == new String("Java")); //false

Q>
String str = " Ineuron\tTechnology\tPrivateLimited\tKnown\tfor\tjava
".strip();
System.out.println(str); // Ineuron Technology PrivateLimited Know for
java

Q> .
if("string".toUpperCase() == "STRING")
{
 System.out.println(true);
}
else
{
 System.out.println(false);
}

Answer: false(comparison happened b/w heap area object and SCP)

Q> .
String, StringBuffer and StringBuilder - all these three classes are final classes.
True or False?
Answer: Yes

Q> .
String str1 = "1";
String str2 = "22";
String str3 = "333";
System.out.println(str1.concat(str2).concat(str3).repeat(3));

Answer: "122".concat("333")
"122333".repeat(3)
122333122333122333

Q>Ronaldo is developing an application in which string concatenation is very frequent.

Which string class do you refer him to use? And also he doesn't need code to be thread safe.
StringBuilder(1.5V)

Q>.
System.out.println("Java"+1000+2000+3000); // "java1000"+2000+3000 =>
"java10002000" + 3000 => "java100020003000"

Q>.
System.out.println(1000+2000+3000+"Java"); //3000+3000+"java" => 6000+"java"

=>"6000java"

Q>.
System.out.println(7.7+3.3+"Java"+3.3+7.7);
//11.0 + "java" + 3.3 +7.7 => "11.0
java"+3.3 => "11.0java3.3" + 7.7 => "11.0java3.37.7"

Q>.
System.out.println("ONE"+2+3+4+"FIVE");
// "ONE2" + 3+4+"FIVE" => "ONE23" + 4
+"FIVE"=> "ONE234+" Five" => "ONE234Five"

Q>.
String s1=" ";
System.out.println(s1.isBlank());
//true
System.out.println(s1.isEmpty());
//false

Q>.
String s2="sachin ramesh tendulkar";
System.out.println(s2.substring(8, 4));

- A. CE
 - B. rame
 - C. in ram
 - D. NullPointerException
 - E. StringIndexOutOfBoundsException
 - F. ArrayIndexOutOfBoundsException
- Answer: E

Q>.
String s1 = new String("JAVA");
String s2 = new String("JAVA");
System.out.println(s1 == s2);
System.out.println(s1.equals(s2));
System.out.println(s1 == s2.intern());
System.out.println(s1.intern() == s2.intern());
System.out.println(s1.intern() == s2);


```
Object [] arr = new Object[4];
for(int i = 1; i <= 3; i++) {
    switch(i) {
        case 1:
            arr[i] = new String("Java");
            break;
        case 2:
            arr[i] = new StringBuilder("Java");
            break;
        case 3:
            arr[i] = new SpecialString("Java");
            break;
    }
}
```

```
class SpecialString extends Object {
    String str;
    SpecialString(String str) {
        this.str = str;
    }
    String toString() {
        return "hexa decimal value"
    }
}
```

arr

0 1 2 3

String

StringBuilder

SpecialString

```
for(Object obj : arr) {
    System.out.println(obj); //null
    System.out.println(obj.hashCode());
}
```

```
String s1 = new String("JAVA");
String s2 = new String("JAVA");
System.out.println(s1 == s2); //false
System.out.println(s1.equals(s2)); //true
System.out.println(s1 == s2.intern()); //false
System.out.println(s1.intern() == s2.intern()); //true
System.out.println(s1.intern() == s2); //false
```